Singly and Doubly Linked Lists in Java

Day-wise DSA notes • Fundamentals, pointer logic, Java implementation, complexity, and practice problems

1. What is a Linked List?

A linked list is a linear data structure made of separate objects called **nodes**. Each node stores data and one or more references (links) to other nodes. Unlike an array, nodes do not need to occupy adjacent memory locations.

Node idea: data + link(s). The **head** reference points to the first node. If head is null, the list is empty.

Why use a linked list?

• Insertions and deletions can be efficient when the required node or its predecessor is already known.
• The list can grow or shrink dynamically.
• Random access is not efficient: reaching index i usually requires walking from the head.

2. Singly Linked List (SLL)

Each node stores data and one reference called **next**. The last node points to null.

```
head
 |
 v
[10 | next] -> [20 | next] -> [30 | null]
```

Basic Java node and traversal

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
    }
}

static void display(Node head) {
    Node current = head;
    while (current != null) {
        System.out.print(current.data + " -> ");
        current = current.next;
    }
    System.out.println("null");
}
```

Pointer tracing: current begins at head. Print current.data, then move current to current.next. The loop stops when current becomes null. Do not move current before printing or the first node will be skipped.

3. Common SLL operations

Insert at beginning: create a node, point its next to the old head, then update head. This is O(1).

```
static Node addFirst(Node head, int value) {
    Node node = new Node(value);
    node.next = head;
    return node;       // new head
}
```

Insert at end: walk to the last node, then attach the new node. Without a tail reference this is O(n).

```
static Node addLast(Node head, int value) {
    Node node = new Node(value);
    if (head == null) return node;

    Node current = head;
    while (current.next != null) {
        current = current.next;
    }
    current.next = node;
    return head;
}
```

4. Delete and search in an SLL

To delete the first node, move head to head.next. To delete a later node, find the node before the target and bypass the target. Edge cases include an empty list and deleting the head.

```
static Node removeFirst(Node head) {
    if (head == null) return null;
    return head.next;
}

static Node deleteValue(Node head, int value) {
    if (head == null) return null;
    if (head.data == value) return head.next;

    Node current = head;
    while (current.next != null &&
           current.next.data != value) {
        current = current.next;
    }

    if (current.next != null) {
        current.next = current.next.next;
    }
    return head;
}

static boolean contains(Node head, int target) {
    for (Node cur = head; cur != null; cur = cur.next) {
        if (cur.data == target) return true;
    }
    return false;
}
```

Important: if the target is absent, deleteValue leaves the list unchanged. If duplicates exist, this version deletes only the first matching value.

5. Doubly Linked List (DLL)

A doubly linked node has three parts: **prev**, data, and **next**. Each node can move forward and backward. The first node's prev is null; the last node's next is null.

```
null <- [10] <-> [20] <-> [30] -> null
         prev     prev
         next     next
```

Node structure and forward/backward traversal

```
class DNode {
    int data;
    DNode prev;
    DNode next;

    DNode(int data) {
        this.data = data;
    }
}

static void forward(DNode head) {
    for (DNode cur = head; cur != null; cur = cur.next) {
        System.out.print(cur.data + " ");
    }
}

static void backward(DNode tail) {
    for (DNode cur = tail; cur != null; cur = cur.prev) {
        System.out.print(cur.data + " ");
    }
}
```

DLL insertion at the beginning

```
static DNode addFirst(DNode head, int value) {
    DNode node = new DNode(value);
    node.next = head;
    if (head != null) {
        head.prev = node;
    }
    return node;
}
```

Notice the two-way link update: node.next points to the old head, and the old head.prev points back to the new node. Missing either assignment breaks the DLL structure.

6. DLL insertion and deletion

If a node reference is already available, unlinking it can be O(1). You must reconnect both sides and update head/tail if the removed node is an endpoint.

```
static DNode removeNode(DNode head, DNode node) {
    if (node == null) return head;

    if (node.prev != null) {
        node.prev.next = node.next;
    } else {
        head = node.next;        // node was head
    }

    if (node.next != null) {
        node.next.prev = node.prev;
    }

    return head;
}
```

This method assumes node belongs to the list. Production code should define how ownership is validated and whether the list maintains a tail reference.

7. SLL vs DLL

Aspect	Singly linked	Doubly linked
Links per node	next	prev and next
Traversal	Forward only	Forward and backward
Memory	Lower overhead	More memory per node
Delete known node	Usually need predecessor	Can unlink directly if links are valid
Implementation	Simpler	More pointer updates / edge cases

8. Complexity quick reference

Operation	SLL	DLL
Access by index	O(n)	O(n)
Search by value	O(n)	O(n)
Insert at head	O(1)	O(1)
Insert at tail	O(n) without tail; O(1) with tail	O(1) with tail
Delete known node	Often O(n) to find predecessor	O(1), given valid node reference

9. Worked problem: Reverse a singly linked list

Input: 10 -> 20 -> 30 -> null
Output: 30 -> 20 -> 10 -> null

Use three references: prev, current, and nextNode. Save the next node before changing current.next, otherwise the remaining list becomes unreachable.

```
static Node reverse(Node head) {
    Node prev = null;
    Node current = head;

    while (current != null) {
        Node nextNode = current.next; // save remainder
        current.next = prev;          // reverse link
        prev = current;               // advance prev
        current = nextNode;           // advance current
    }
    return prev;
}
```

Time: O(n), because each node is processed once. Extra space: O(1), because only a few references are used.

10. Worked problem: Find the middle node

Use slow and fast pointers. Slow moves one step; fast moves two. When fast reaches the end, slow is at the middle. For an even number of nodes, this implementation returns the second middle.

```
static Node middle(Node head) {
    Node slow = head;
    Node fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

Time: O(n). Extra space: O(1). The condition order matters: check fast != null before reading fast.next.

11. Common mistakes and practice

• Forgetting to return the new head after inserting or deleting the first node.
• Writing current = current.next before processing current.
• Forgetting to update prev links in a DLL.
• Dereferencing head.next when head is null.
• Losing the rest of a list by changing a next pointer before saving it.

Practice in order: (1) count nodes, (2) find the nth node, (3) insert at a given position, (4) delete all occurrences of a value, (5) reverse a DLL, (6) check whether a list is a palindrome.

Interview Aspect: Explain that linked lists provide sequential access, not constant-time indexed access. State assumptions clearly—especially whether a tail reference or a direct node reference is available—before claiming O(1).